Assignment-1.4

2303A51832
B_26

1	Lab 1: Environment Setup – *GitHub Copilot and VS Code Integration + Understanding AI-assisted Coding Workflow* **Lab Objectives:** ● To install and configure GitHub Copilot in Visual Studio Code. ● To explore AI-assisted code generation using GitHub Copilot. ● To analyze the accuracy and effectiveness of Copilot's code suggestions. ● To understand prompt-based programming using comments and code context **Lab Outcomes (LOs):** After completing this lab, students will be able to: ● Set up GitHub Copilot in VS Code successfully. ● Use inline comments and context to generate code with Copilot. ● Evaluate AI-generated code for correctness and readability. ● Compare code suggestions based on different prompts and programming styles. Task 0 ● Install and configure GitHub Copilot in VS Code. Take screenshots of each step. Expected Output ● Install and configure GitHub Copilot in VS Code. Take screenshots of each step. Task 1: AI-Generated Logic Without Modularization (Prime Number Check Without Functions)	Week1 - Monday

fibonacci.py factorial.py prime num.py

C: > Users > prade > OneDrive > Pradeep > prime num.py > ...

```python
# AI-Generated Prime Number Check Without Functions
# This script checks if a given number is prime without using any functions

# Get input from user
num = int(input("Enter a number to check if it's prime: "))

# Initialize flag
is_prime = True

# Check if number is less than or equal to 1
if num <= 1:
    is_prime = False
else:
    # Check for factors from 2 to sqrt(num)
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            is_prime = False
            break

# Output the result
if is_prime:
    print(f"{num} is a prime number.")
else:
    print(f"{num} is not a prime number.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

```
C:\Users\prade>C:/Users/prade/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/prade/OneDrive/Pradeep/prime num.py"
Enter a number to check if it's prime: 2
2 is a prime number.
```

Here we are checking whether the given number is prime or not.

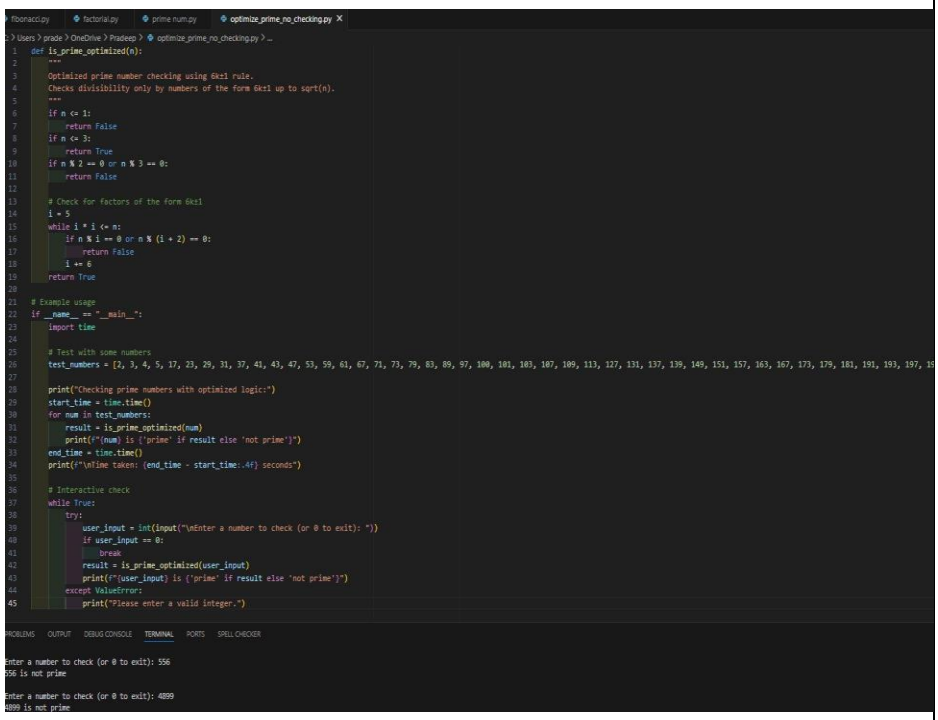
- **Scenario**
  - You are developing a **basic validation script** for a numerical learning application.

- **Task Description**

  Use GitHub Copilot to generate a Python program that:
  - Checks whether a given number is **prime**
  - Accepts user input
  - Implements logic **directly in the main code**
  - Does **not** use any user-defined functions

- **Expected Output**
  - Correct prime / non-prime result
  - Screenshots showing Copilot-generated code suggestions
  - Sample inputs and outputs

Task 2: Efficiency & Logic Optimization (Cleanup)



- **Scenario**
  The script must handle larger input values efficiently.

- **Task Description**
  Review the Copilot-generated code from Task 1 and improve it by:
  - Reducing unnecessary iterations
  - Optimizing the loop range (e.g., early termination)
  - Improving readability
  - Use Copilot prompts like:
    - *"Optimize prime number checking logic"*
    - *"Improve efficiency of this code"*

Hint:
Prompt Copilot with phrases like
"optimize this code", *"simplify logic"*, or *"make it more readable"*

- **Expected Output**
  - Original and optimized code versions
  - Explanation of how the improvements reduce time complexity

Task 3: Modular Design Using AI Assistance (Prime Number Check Using Functions)

fibonacci.py | factorial.py | prime num.py | optimize_prime_no_checking.py | prime check in functions.py

C: > Users > prade > OneDrive > Pradeep > prime check in functions.py > ...

```python

# AI-assisted Python program for prime number checking using functions
# This program defines a function to check if a number is prime and returns a Boolean value

def is_prime(number):
    """
    Check if a given number is prime.

    Args:
    number (int): The number to check for primality.

    Returns:
    bool: True if the number is prime, False otherwise.
    """
    # Numbers less than or equal to 1 are not prime
    if number <= 1:
        return False

    # 2 and 3 are prime numbers
    if number <= 3:
        return True

    # Check if the number is divisible by 2 or 3
    if number % 2 == 0 or number % 3 == 0:
        return False

    # Check for factors from 5 to sqrt(number), skipping even numbers and multiples of 3
    i = 5
    while i * i <= number:
        if number % i == 0 or number % (i + 2) == 0:
            return False
        i += 6

    # If no factors found, the number is prime
    return True
```

```python
# Main function to test the is_prime function
def main():
    """
    Main function to demonstrate the prime checking functionality.
    """
    # Test cases
    test_numbers = [2, 3, 4, 5, 17, 23, 29, 31, 37, 41, 43, 47]

    print("Prime number checker (AI-assisted):")
    for num in test_numbers:
        result = is_prime(num)
        status = "is prime" if result else "is not prime"
        print(f"{num} {status}")

    # Interactive input
    while True:
        try:
            user_input = int(input("\nEnter a number to check (or 0 to exit): "))
            if user_input == 0:
                break
            result = is_prime(user_input)
            status = "is prime" if result else "is not prime"
            print(f"{user_input} {status}")
        except ValueError:
            print("Please enter a valid integer.")

# Run the main function if the script is executed directly
if __name__ == "__main__":
    main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1

```
Enter a number to check (or 0 to exit): 3
3 is prime

Enter a number to check (or 0 to exit): 1
1 is not prime

Enter a number to check (or 0 to exit):
```

Here we find prime numbers using functions

- **Scenario**

  The prime-checking logic will be reused across multiple modules.

❖ **Task Description**

Use GitHub Copilot to generate a function-based Python program that:

- Uses a user-defined function to check primality
- Returns a Boolean value
- Includes meaningful comments (AI-assisted)

❖ **Expected Output**

- Correctly working prime-checking function
- Screenshots documenting Copilot's function generation
- Sample test cases and outputs

Task 4: Comparative Analysis –With vs Without Functions

fibonacci.py | factorial.py | prime num.py | optimize_prime_no_checking.py | prime check in functions.py | algoritham.py.py | comparative.py

C: > Users > prade > OneDrive > Pradeep > comparative.py > is_prime_function

```python
# Comparative Analysis: Prime Number Checking With vs Without Functions
# This program compares the efficiency of prime checking logic implemented with and without user-defined functions
# AI-generated code

import time

# Function-based implementation
def is_prime_function(n):
    """
    Check if a number is prime using a function.
    Returns True if prime, False otherwise.
    """
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True
```

fibonacci.py | factorial.py | prime num.py | optimize_prime_no_checking.py | prime check in functions.py | algoritham.py.py | comparative.py

C: > Users > prade > OneDrive > Pradeep > comparative.py > is_prime_function

```python
# Non-function implementation (inline logic)
def check_primes_without_function(numbers):
    """
    Check a list of numbers for primality without using a separate function.
    Returns a list of results.
    """
    results = []
    for num in numbers:
        # Inline prime checking logic
        is_prime = True
        if num <= 1:
            is_prime = False
        elif num > 3:
            if num % 2 == 0 or num % 3 == 0:
                is_prime = False
            else:
                i = 5
                while i * i <= num:
                    if num % i == 0 or num % (i + 2) == 0:
                        is_prime = False
                        break
                    i += 6
        results.append(is_prime)
    return results

# Test data
test_numbers = [2, 3, 4, 5, 17, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163,

print("Comparative Analysis: Prime Checking With vs Without Functions")
print("=" * 60)

# Test with functions
print("\nTesting with function-based approach:")
start_time = time.time()
function_results = [is_prime_function(num) for num in test_numbers]
function_time = time.time() - start_time
print(f"Time taken: {function_time:.6f} seconds")

# Test without functions
print("\nTesting without function-based approach:")
start_time = time.time()
inline_results = check_primes_without_function(test_numbers)
inline_time = time.time() - start_time
print(f"Time taken: {inline_time:.6f} seconds")
```

```python
# Verify results are the same
if function_results == inline_results:
    print("\n✓ Results are identical - both approaches produce the same output")
else:
    print("\n✗ Results differ - there might be an error in implementation")

# Performance comparison
print("Performance Comparison:")
print(f"Function-based time: {function_time:.6f} seconds")
print(f"Inline time: {inline_time:.6f} seconds")
if function_time < inline_time:
    print("Function-based approach is faster")
elif inline_time < function_time:
    print("Inline approach is faster")
else:
    print("Both approaches have similar performance")

# Display some results
print("Sample Results:")
for i in range(min(10, len(test_numbers))):
    num = test_numbers[i]
    print(f"{num}: {'Prime' if function_results[i] else 'Not Prime'}")

print("\nNote: Function-based code is more modular, reusable, and easier to maintain,")
print("while inline code might have slight performance advantages for simple cases.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 2

```
Enter a number to check (or 0 to exit): 5
5 is prime

Enter a number to check (or 0 to exit): 1
1 is not prime
```

- **Scenario**
  You are participating in a technical review discussion.

- **Task Description**
  Compare the Copilot-generated programs:
  - Without functions (Task 1)
  - With functions (Task 3)
  - Analyze them based on:
  - Code clarity
  - Reusability
  - Debugging ease
  - Suitability for large-scale applications

- **Expected Output**
  Comparison table or short analytical report

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to Prime Checking)

fibonacci.py | factorial.py | prime num.py | optimize_prime_no_checking.py | prime check in functions.py | algoritham.py.py

C: > Users > prade > OneDrive > Pradeep > algoritham.py.py > ...

```python
#generate a python on Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to Prime Checking)
def fibonacci_iterative(n):
    """
    Calculate Fibonacci number using an iterative approach.

    Args:
    n (int): The position in the Fibonacci sequence.

    Returns:
    int: The Fibonacci number at position n.
    """
    if n <= 0:
        return 0
    elif n == 1:
        return 1

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
def fibonacci_recursive(n):
    """
    Calculate Fibonacci number using a recursive approach.

    Args:
    n (int): The position in the Fibonacci sequence.

    Returns:
    int: The Fibonacci number at position n.
    """
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

```python
# Example usage
if __name__ == "__main__":
    n = 10  # Change this value to compute a different Fibonacci number

    print(f"Fibonacci number at position {n} (Iterative): {fibonacci_iterative(n)}")
    print(f"Fibonacci number at position {n} (Recursive): {fibonacci_recursive(n)}")
    print("Please enter a valid integer.")
    # Performance comparison
    import time
    n_large = 30  # Larger value for performance comparison
    start_time = time.time()
    print(f"Fibonacci number at position {n_large} (Iterative): {fibonacci_iterative(n_large)}")
    end_time = time.time()
    print(f"Iterative approach took {end_time - start_time:.6f} seconds")
    start_time = time.time()
    print(f"Fibonacci number at position {n_large} (Recursive): {fibonacci_recursive(n_large)}")
    end_time = time.time()
    print(f"Recursive approach took {end_time - start_time:.6f} seconds")
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | SPELL CHECKER 1

```
Enter a number to check (or 0 to exit): 2
2 is prime

Enter a number to check (or 0 to exit): 3
3 is prime

Enter a number to check (or 0 to exit): 55
55 is not prime
```

- **Scenario**
  Your mentor wants to evaluate how AI handles **alternative logical strategies**.

- **Task Description**
  Prompt GitHub Copilot to generate:
  - A **basic divisibility check** approach
  - An **optimized approach** (e.g., checking up to √n)

	❖ **Expected Output** ➢ Two correct implementations ➢ Comparison discussing: ▪ Execution flow ▪ Time complexity ▪ Performance for large inputs ▪ When each approach is appropriate **Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.**	